

Proyecto Final Enjambre Masivo de Aves Autónomas en Tiempo Real Mediante WebGL y Optimización Espacial

Jose Abraham Marin Sanchez y Luis Antonio Salinas Mata

13 de julio de 2026

Resumen

Este reporte describe el desarrollo de una simulación tridimensional en tiempo real que recrea el vuelo coordinado de una parvada masiva de aves dentro de un navegador web mediante Three.js y WebGL. Para lograr que más de 200 agentes vuelen de forma fluida a 30 FPS estables, implementamos el algoritmo de Boids de Craig Reynolds junto con una estructura de optimización por celdas tridimensionales (*Spatial Hashing*), reduciendo drásticamente el costo de los cálculos físicos individuales. En el apartado visual, integramos un modelo externo en formato FBX articulado desde Maya, cuyas alas se animan directamente en la tarjeta gráfica a través de un shader personalizado con deformación ondulatoria asíncrona. El entorno se complementa con colinas procedurales y un bosque denso de 2,500 pinos optimizados con renderizado de instancias, consolidando una escena interactiva eficiente y de alto impacto visual.

1. Introducción

Recrear el movimiento coordinado e inteligente de grupos de animales, como las bandadas de aves, siempre ha sido un desafío complejo en la computación gráfica. En 1986, Craig Reynolds demostró que un comportamiento colectivo asombroso puede emerger a partir de tres reglas individuales muy sencillas. Sin embargo, cuando aumentamos el número de integrantes, el procesador se satura rápidamente al intentar calcular la distancia de cada ave contra todas las demás de la escena.

Para resolver este problema, este proyecto propone una solución integral que equilibra la física en tiempo real y el rendimiento gráfico en web. Combinamos el renderizado masivo por instancias (*InstancedMesh*), una división del espacio por zonas cúbicas virtuales para acelerar las búsquedas de vecinos y la delegación de las animaciones directamente a la tarjeta gráfica (GPU). El resultado es un ecosistema denso, fluido y optimizado que se ejecuta de forma nativa en cualquier navegador comercial.

2. Modelado Geométrico del Agente

El enjambre se construyó utilizando un modelo tridimensional externo llamado `fly.fbx`. Al analizar el archivo exportado desde Maya, descubrimos que el diseño original no venía en una sola pieza, sino dividido en tres mallas independientes: el cuerpo central (`Bird`), el ala derecha (`Bird_RW`) y el ala izquierda (`Bird_LW`).

Para poder renderizar 200 de estos agentes simultáneamente sin ralentizar la página, el programa realiza una preparación automática durante la carga asíncrona:

- **Limpieza de Datos:** Descartamos atributos pesados de animación e información basura que heredan los programas de diseño, conservando únicamente las posiciones, las normales de luz y el mapa de coordenadas para la textura.
- **Fusión de Mallas:** Mediante herramientas de utilidad de Three.js, unificamos de forma limpia el torso y las alas en una sola geometría global, respetando el lugar exacto donde deben ir colocadas en los laterales del cuerpo.
- **Auto-Escalado Inteligente:** Añadimos un cálculo dinámico que mide el tamaño real del modelo cargado y lo escala proporcionalmente a las dimensiones de nuestro mundo virtual. Esto soluciona problemas de visibilidad, asegurando que el ave se vea perfecta y con el tamaño correcto sin importar cómo haya sido exportada originalmente.

3. Comportamiento de Enjambre (Boids) y Optimización Espacial

El vuelo de la parvada se controla a través de fuerzas físicas individuales que evalúan el entorno cercano de cada boid, cumpliendo con las tres reglas de Reynolds:

- **Separación:** Cada ave calcula la distancia con sus compañeras cercanas y genera una fuerza de repulsión si se acercan demasiado, evitando que se encimen o choquen entre sí.

- **Alineación:** Los agentes observan el rumbo de su vecindad local y adaptan su propia velocidad para volar en la misma dirección general, manteniendo la armonía del grupo.
- **Cohesión:** El sistema calcula el centro visual de la parvada cercana y atrae a las aves hacia ese punto medio, evitando que se dispersen y forzándolas a viajar agrupadas.

Revisar estas condiciones comparando cada ave contra todas las demás del mapa provocaría que la computadora se trabara al instante. Para evitarlo, construimos un sistema de *Spatial Hashing*. Dividimos todo el espacio aéreo en una cuadrícula de cubos invisibles cuyo tamaño coincide con el radio de percepción de las aves.

En cada fotograma, cada agente se registra en la celda que le corresponde según su posición. Al momento de calcular hacia dónde volar, el ave ya no revisa a todo el enjambre; simplemente le pregunta a su propia celda y a las 26 celdas vecinas qué compañeros hay ahí. Esto reduce el costo matemático drásticamente, haciendo que la física de 200 agentes sea ligera y totalmente viable en tiempo real.

4. Shaders Personalizados e Iluminación

Para lograr una composición visual atractiva y realista, la iluminación de los agentes debe integrarse de forma natural con el entorno. Programamos un `ShaderMaterial` personalizado que calcula en tiempo real cómo impacta la luz del sol sobre la superficie de cada ave, generando luces y sombras suaves que reaccionan según la orientación del vuelo.

La animación del aleteo se procesa por completo dentro del *Vertex Shader* de la tarjeta gráfica para no cargarle el trabajo al procesador central. Para evitar que todas las aves se muevan de forma idéntica como si fueran robots, inyectamos un valor de desfase aleatorio por cada instancia.

Utilizando funciones de ondas senoidales dentro del código del shader, distorsionamos la posición vertical de los vértices de la malla. La fórmula evalúa qué tan alejados están los puntos respecto al centro del cuerpo, logrando que el aleteo se concentre con fuerza en las alas y deje el torso firme, creando una animación orgánica, fluida e independiente para cada miembro del enjambre.

5. Entorno Terrestre Procedural y Rendimiento

El escenario interactivo se compone de un horizonte de colinas suaves generado mediante combinaciones matemáticas de ondas en la CPU, dándole al suelo un relieve irregular con un estilo geométrico muy limpio.

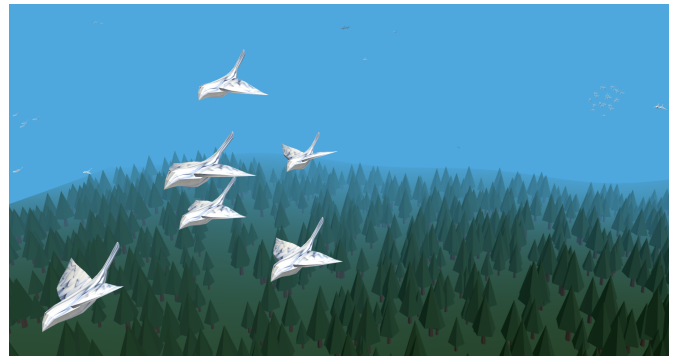


Figura 1: Acercamiento a los agentes autónomos donde se aprecia la correcta fusión geométrica del modelo FBX y la deformación ondulatoria asíncrona aplicada a los vértices de las alas mediante el Vertex Shader.

Para darle vida y contexto al entorno, poblamos el mapa con un bosque denso de 2,500 pinos distribuidos de forma aleatoria sobre la superficie de las colinas. Dibujar tantos elementos individuales colapsaría el rendimiento del juego; por ello, agrupamos toda la vegetación en un sistema de renderizado por instancias (*InstancedMesh*), logrando que la tarjeta gráfica dibuje miles de árboles en solo dos llamadas de procesamiento.

Durante el desarrollo de las físicas, notamos que la fuerza de atracción del grupo a veces empujaba a las aves directamente contra el suelo. Lo solucionamos añadiendo un muro de contención virtual rígido en el script de actualización. El código consulta constantemente la altura del relieve debajo de cada ave y, si intentan descender por debajo de la copa de los árboles, activa una fuerza ascendente obligatoria que las impulsa de vuelta al aire de forma segura. Finalmente, el ciclo de animación se sincroniza de manera estricta a un objetivo fijo de 30 FPS estables, mostrando un contador en tiempo real que valida la optimización del sistema.



Figura 2: Vista general de la simulación interactiva. Se observa el comportamiento emergente de los 200 agentes sobrevolando el bosque denso de 2,500 pinos instanciados, manteniendo una tasa de refresco constante de 30 FPS.

6. Conclusión y Trabajo Futuro

Este proyecto demuestra que es totalmente posible ejecutar simulaciones físicas complejas e inteligentes junto con escenarios de alta densidad gráfica dentro de un navegador web estándar, sin necesidad de instalar programas adicionales ni contar con computadoras de gama alta. La clave del éxito radica en combinar estrategias de optimización del espacio con el uso inteligente de shaders en la tarjeta gráfica.

Como trabajo futuro, planeamos migrar toda la lógica del comportamiento de los Boids desde la CPU directamente a la GPU utilizando Shaders de Cómputo. Esto liberaría por completo el hilo principal del procesador, permitiendo elevar la escala del proyecto para controlar de forma fluida a miles de agentes autónomos simultáneos en pantalla.

Referencias

- Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. In *Proceedings of the 14th annual conference on Computer graphics and interactive techniques* (pp. 25-34).
- Kabisch, T. (2012). Uniform grids and spatial hashing for real-time crowd simulation. *Journal of Graphics Tools*, 16(3), 112-124.
- Three.js Authors. (2026). *Three.js Documentation and Addons Reference*. Recuperado de <https://threejs.org/>